

Cahier des Charges — NarrativePath (version finale)

1. Description générale

Objectifs

Concevoir et développer une application web de fiction interactive, accessible sans création de compte, qui adapte automatiquement le contenu narratif à l'âge de l'utilisateur et lui permet de personnaliser son expérience (prénom, style narratif), tout en restant entièrement sécurisée par construction (pas de génération de contenu non contrôlée).

Public cible

Deux tranches d'âge distinctes : les enfants de 6 à 12 ans (vocabulaire simple, contenu toujours positif et rassurant) et les adolescents de 13 à 18 ans (récits plus riches, enjeux plus nuancés, sans contenu sensible).

Plus-value

Contrairement à un livre interactif classique (contenu figé et répétitif) ou à un outil d'IA générative non contrôlé (type AI Dungeon, contenu imprévisible), NarrativePath simule un comportement d'IA personnalisée tout en garantissant que tout le contenu affiché a été écrit et validé à l'avance, par tranche d'âge.

Contexte

Un parent souhaite proposer une histoire interactive à son enfant, mais les solutions existantes posent un problème de fond : soit le contenu est limité et répétitif (livres interactifs), soit il n'est pas contrôlé (outils d'IA générative grand public). NarrativePath répond à la question : comment proposer une narration interactive personnalisée et sécurisée, pour des publics allant de 6 à 18 ans ?

2. Liste exhaustive des fonctionnalités

- **Sélection du profil d'âge** (6-12 ans / 13-18 ans), demandée à chaque ouverture de l'application, avant toute autre étape.
- **Formulaire de personnalisation** : saisie du prénom de l'utilisateur et choix d'un style narratif parmi trois (aventure, humour, fantastique).
- **6 histoires indépendantes**, une par combinaison âge × style, chacune avec sa propre structure de nœuds (nombre de branches et de fins propre à chaque histoire).
- **Moteur narratif à base de nœuds** : chaque nœud contient un texte et 2 à 3 choix (sauf les nœuds de fin, qui n'ont aucun choix), avec navigation entre les nœuds selon les choix de l'utilisateur.
- **Personnalisation du texte** : le prénom saisi est intégré directement dans le texte des nœuds.

- **Sauvegarde de la progression** via `localStorage` : une histoire interrompue (rechargement de page) peut être reprise si la même combinaison âge/style est resélectionnée et que la position sauvegardée n'est pas déjà une fin.
- **Bouton "Recommencer"**, toujours visible, qui réinitialise entièrement l'expérience (âge, prénom, style et progression sont redemandés).
- **Illustrations générées par IA** : une image SVG par style narratif, affichée en haut de l'écran d'histoire, et un logo SVG sur l'écran d'accueil.
- **Effet de transition (fade-in)** à l'affichage de chaque écran (sélection d'âge, formulaire, nœud narratif).
- **Interface responsive**, construite uniquement avec Tailwind CSS.

3. Stack technique — avec justification

Technologie	Rôle	Justification
Vite.js	Outil de build obligatoire du projet	Fournit un serveur de développement avec rechargement à chaud (HMR) quasi instantané, et génère via <code>npm run build</code> un dossier <code>dist/</code> optimisé et minifié, prêt à être déployé sans configuration supplémentaire.
JavaScript Vanilla	Langage applicatif, sans framework	Garde le code simple, lisible et entièrement compréhensible sans dépendre d'un framework (React, Vue), ce qui correspond à l'objectif pédagogique du projet.
Tailwind CSS	Mise en forme de l'interface	Permet d'écrire un style cohérent directement via des classes utilitaires dans le JavaScript, sans maintenir de fichiers CSS séparés pour chaque composant.
JSON mock	Source des données narratives	Simule ce qu'une génération de contenu par IA produirait, sans dépendre d'une API externe réelle ni d'un backend, tout en gardant un contenu entièrement maîtrisé et sécurisé.
localStorage	Persistance côté navigateur	Seule forme de sauvegarde autorisée par le projet (aucune base de données) : stocke uniquement la progression de l'histoire en cours, localement, sans serveur.

Aucune API externe ni base de données n'est utilisée, conformément au périmètre du projet.

4. Assets multimédia générés par IA

Tous les assets visuels du projet sont des illustrations vectorielles (SVG) générées par l'assistant IA Claude, sans outil de génération d'image externe, et stockées dans `src/assets/` :

Fichier	Outil	Description
<code>adventure.svg</code>	Claude	Bannière illustrant le style "aventure" : forêt, chemin sinueux, coffre au trésor, boussole.

funny.svg	Claude	Bannière illustrant le style "humour" : couleurs vives, confettis, visage souriant, ton ludique.
fantasy.svg	Claude	Bannière illustrant le style "fantastique" : ciel étoilé, lune, château onirique, lueur magique.
logo.svg	Claude	Logo du projet "NarrativePath", affiché sur l'écran d'accueil.

Ces illustrations sont affichées dynamiquement selon le style narratif choisi par l'utilisateur (écran d'histoire) et sur l'écran de sélection d'âge (logo).

5. Organisation du projet

```
src/
├── data/
│   ├── stories.js           → sélection de l'histoire selon âge + style (selectStory)
│   └── stories/
│       ├── 6-12_adventure.json
│       ├── 6-12_funny.json
│       ├── 6-12_fantasy.json
│       ├── 13-18_adventure.json
│       ├── 13-18_funny.json
│       └── 13-18_fantasy.json → 6 histoires indépendantes (1 par combinaison âge × style)
├── engine/
│   ├── narrativeEngine.js   → moteur narratif : navigation, historique, reprise de progression
│   └── buildText.js         → composition du texte final affiché (remplacement du prénom)
├── ui/
│   ├── ageSelector.js       → écran de sélection du profil d'âge
│   ├── profileForm.js       → écran de saisie du prénom et du style narratif
│   └── render.js            → affichage du nœud courant (texte, choix, illustration, bouton Re
├── assets/                  → illustrations SVG générées par IA (par style + logo)
├── documents/               → documentation du projet
│   ├── Cahier des Charges Projet Narrativ.txt (ce document)
│   ├── Méthode MoSCoW NarrativePath.txt
│   ├── Prompts.md           → prompts IA sécurisés simulés, par tranche d'âge
│   └── Journal-de-prompting.md → traçabilité du pilotage de l'assistant IA
├── main.js                  → point d'entrée, orchestration des écrans
└── style.css                → import Tailwind + transition fade-in
```

6. Méthodologie de réalisation

Le projet a été développé en pilotant un assistant IA (**Claude**, dans l'environnement VS Code) de façon itérative : chaque fonctionnalité a été demandée via un prompt, le résultat a été testé (lancement de `npm run dev` / `npm run build`, vérification visuelle, scripts de validation des données),

puis corrigé ou affiné par des prompts de suivi en cas d'écart avec le besoin réel. Cette démarche est détaillée dans [Journal-de-prompting.md](#) .